National University of Computer and Emerging Sciences

DSA Assignment 2 Phase 1 Report

Student Name: Ahmad Ali **Roll Number: 25i-0817** **Section: BS CS E**

Selected Data Structures and Justifications

- **Stack Representation**

  I select dynamic array because:

  1. It stores elements in continuous memory for instant access.
  2. Push and pop run in strict O(1) time without pointer overhead.
  3. It dynamically doubles memory when full so no operation fails.

- **Queue Representation**

  I select circular array because:

  1. It connects front and rear pointers using modulo arithmetic.
  2. Add and delete run in O(1) time without moving or shifting elements.
  3. It enforces bounded capacity to protect memory from uncontrolled growth.

- **Empty Queue versus Full Queue Identification**

  I select dedicated count variable because:

  1. It directly tracks the active element count at any point.
  2. It tells full from empty without wasting a reserved slot in the array.
  3. It provides an instant O(1) answer for queue size queries.

- **Bounded Pipeline Policy**

  I select reject new jobs when full because:

  1. It stops memory overflow when maximum queue capacity is reached.
  2. It prompts the user to process pending tasks before adding new ones.
  3. It maintains application stability under heavy edit requests.

- **Pipeline Job Cancellation Policy**

  I select in place cancellation marker flag because:

  1. It flags the job as cancelled without shifting other elements.
  2. It avoids rebuilding or reordering the queue array.
  3. The pipeline safely skips the task when it reaches the front during execution.

- **Pending Jobs with Undo and Redo Policy**

  I select queued execution when pipeline is active because:

  1. It preserves the exact chronological order of editing actions.
  2. It ensures an undo operates on the image only after earlier queued jobs finish.
  3. It runs immediately whenever the pipeline queue is empty.

- **Bucket Fill Traversal**

  I select non recursive Stack fill and Queue fill because:

  1. Stack fill explores deep neighbor branches along one path first.
  2. Queue fill spreads outward evenly in concentric radial waves.
  3. Explicit data structures prevent system call stack overflow on large images.

- **History Management Engine**

  I select dual stacks with deep copy snapshots because:

  1. It isolates undo history from redo history cleanly.
  2. Any fresh edit immediately clears the redo stack to keep history linear.
  3. Full matrix snapshots guarantee bit for bit image recovery upon undo.

Operation Complexity and Analysis

Operation Name	Time	Space	Reason
Load Image	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	I use binary stream reading to parse image header and stream pixels directly into grid.
Crop Image	$\mathcal{O}(W' \times H')$	$\mathcal{O}(W' \times H')$	I use nested loops to copy pixels strictly within the selected crop bounding box.
Rotate 90 and 270 deg	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use coordinate transposition which maps source rows into target columns.
Rotate 180 deg	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	I use symmetric in place pointer swapping between opposite halves of the image grid.
Horizontal Flip	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	I use in place swapping of symmetric column pixels along each row.
Vertical Flip	$\mathcal{O}(H)$	$\mathcal{O}(1)$	I use direct row pointer swaps which avoids touching individual pixel data.
Brightness Adjustment	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	I use direct arithmetic addition and clamp every value between 0 and 255.
Grayscale Conversion	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	I use fixed point luminance weighting to compute average intensity per pixel.
Bilinear Resize	$\mathcal{O}(W' \times H')$	$\mathcal{O}(W' \times H')$	I use four point weighted interpolation for each target dimension pixel.
Box Blur (3×3)	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use standard matrix kernel where 9 neighbor sums take O(1) time per pixel.
Bucket Fill (Stack)	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use an explicit stack to visit every connected matching pixel once.
Bucket Fill (Queue)	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use a circular queue to expand outward layer by layer in level order.
Stack Push, Pop, Top	$\mathcal{O}(1)$	$\mathcal{O}(1)$	I use direct indexing at the stack top position without moving elements.
Queue Add, Delete, Front	$\mathcal{O}(1)$	$\mathcal{O}(1)$	I use circular modulo arithmetic to advance front and rear pointers instantly.
Undo and Redo Operation	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use deep copy restoration to pop snapshot state from stack to canvas.

Core Operational Guarantees

- **Strict Zero Shifting:** The circular queue never shifts elements upon `deleteQueue()`. Pointers advance via $(rear+1) \pmod{capacity}$ and $(front+1) \pmod{capacity}$.
- **Stack Safety and Growth:** The stack dynamically doubles memory when full, ensuring no push fails, and checks empty status to prevent underflow crashes.
- **Four Connectivity Rule:** Bucket fill inspects only up, down, left, and right neighbors. Diagonal pixels are never accessed.
- **Identical Output Proof:** Both Stack fill and Queue fill produce bit for bit identical output images for identical seed coordinates and replacement colors.

Testing Evidence and Verification Results

Component Tested	Cases Verified	Result
Stack (stackType)	Pop and top on empty stack; 1000 item capacity; repeated push pop cycles; stack reset; deep copy constructor and assignment.	**PASS**
Queue (queueType)	Delete and front on empty queue; full queue rejection; circular wrap around; drain and re add; deep copy constructor.	**PASS**
Undo and Redo Engine	Multiple consecutive undos; multiple consecutive redos; clearing redo stack after new action; bit for bit image recovery.	**PASS**
Bucket Fill Engine	Single pixel fill; full image fill; irregular boundary fill; disconnected regions; identical Stack and Queue outputs; same color immediate exit.	**PASS**
Processing Pipeline	Bounded queue rejection; process next and all jobs; in place job cancellation without shifting; queue clearing.	**PASS**

Visual Verification: Bucket Fill and Test Suite Execution

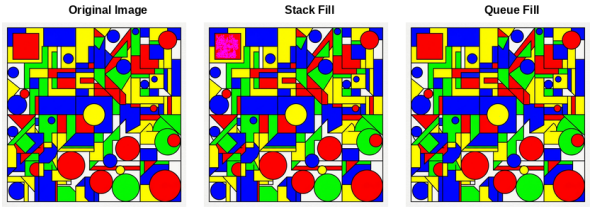


Figure 1: Side by side visual proof: Stack fill (middle) and Queue fill (right) yield identical results.

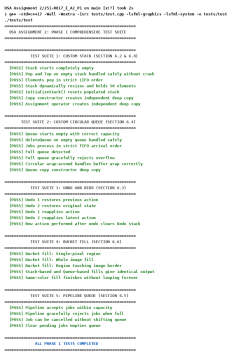


Figure 2: Automated test suite output verifying 100 percent pass across all required edge cases.